

Лекция 4. Нейронные сети

Часть 2

Дополнительные главы машинного обучения

Автор занятия: Никитин Илья Сергеевич

Образовательная программа
«Машинное обучение в цифровом продукте»

Оглавление

1) ResNet, BatchNorm и Dropout

2) Свёртки и свёрточные сети (CNN)

3) Рекуррентные сети (RNN)

4) Долгая краткосрочная память (LSTM)

Глубина и масштаб градиента

При движении к ранним слоям градиент многократно умножается на матрицы весов и производные активаций:

$$\nabla_{h^{(0)}} \ell = J_1^\top J_2^\top \cdots J_L^\top \nabla_{h^{(L)}} \ell.$$

- Если преобразования сжимают сигнал, ранние слои получают всё меньший градиент.
- Если они слишком сильно его растягивают, градиент может взрываться.
- Насыщение активаций также уменьшает локальные производные.

Масштаб начальных весов определяет, с какими активациями и градиентами начнётся обучение.

Почему начальные веса должны различаться

У двух одинаково инициализированных скрытых нейронов одинаковые выходы. Если совпадают и их связи со следующим слоем, одинаковыми будут и градиенты.

$$w_1 = w_2, \quad b_1 = b_2 \quad \implies \quad h_1(x) = h_2(x).$$

После обновления нейроны остаются одинаковыми и продолжают учить один и тот же признак.

- Независимые случайные веса нарушают эту симметрию.
- Смещения при этом часто можно начать с нуля.
- Случайность нужна вместе с правильно выбранным масштабом весов.

Инициализация Xavier и He

Для $z_j = \sum_{i=1}^{d_{\text{in}}} W_{ji} h_i$ при независимых центрированных весах:

$$\text{Var}(z_j) \approx d_{\text{in}} \text{Var}(W_{ji}) \mathbb{E}[h_i^2].$$

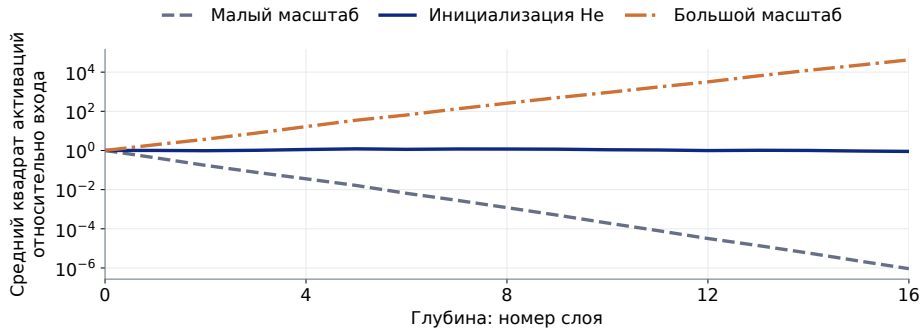
Чтобы масштаб сигнала не менялся резко от слоя к слою, учитываем число входов и активацию.

- **Xavier:** $\text{Var}(W_{ji}) = 2/(d_{\text{in}} + d_{\text{out}})$, компромисс для прямого и обратного проходов при близкой к линейной активации.
- **He:** $\text{Var}(W_{ji}) = 2/d_{\text{in}}$ для ReLU. Множитель 2 компенсирует обнуление примерно половины симметричного сигнала.

Это начальный выбор масштаба, основанный на приближённых предположениях.

Glorot, Bengio, Understanding the Difficulty of Training Deep Feedforward Neural Networks, 2010; He et al., Delving Deep into Rectifiers, 2015.

Как начальный масштаб меняется по глубине



Слишком малые веса гасят активации, слишком большие усиливают их от слоя к слою. Инициализация He учитывает ReLU и помогает сохранить масштаб.

Здесь показан прямой проход случайных ReLU-сетей до обучения, без смещений. Кривые усреднены по нескольким инициализациям.

ResNet: остаточная связь

В остаточном блоке к выходу обучаемой ветви прибавляют вход:

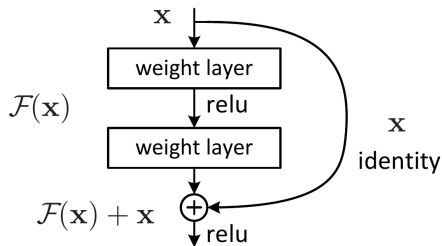
$$u = h + F(h).$$

Блок учит **поправку** к исходному представлению.

Если представление уже подходит, ветвь может приблизить $F(h) \approx 0$ и сохранить его в сумме.

На рисунке после суммы стоит ReLU:

$h_{\text{next}} = \text{ReLU}(u)$. Складываемые тензоры имеют одинаковую форму.



He et al., Deep Residual Learning for Image Recognition, рис. 2.

Почему остаточная связь помогает обучению

Для суммы $u = h + F(h)$ градиент имеет два слагаемых:

$$\nabla_h \ell = \underbrace{\nabla_u \ell}_{\text{прямой путь}} + \underbrace{J_F(h)^\top \nabla_u \ell}_{\text{через обучаемую ветвь}}.$$

- Часть градиента проходит к входу без умножения на производные внутри F .
- Глубокой сети проще сохранить полезное представление и постепенно уточнять его.
- Дополнительные слои могут учить небольшую поправку вместо полного преобразования заново.

Формула записана для суммы u , до следующей активации. Residual-связи облегчают оптимизацию глубоких сетей.

BatchNorm: нормализация внутри сети

Для одного признака $z_1, \dots, z_B$ в обучающем мини-батче:

$$\mu_B = \frac{1}{B} \sum_i z_i, \quad \sigma_B^2 = \frac{1}{B} \sum_i (z_i - \mu_B)^2,$$

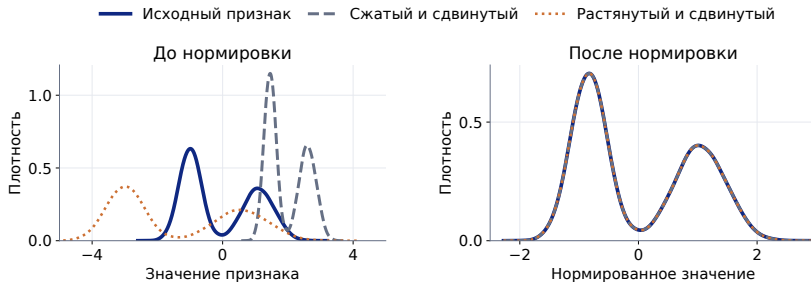
$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \text{BN}(z_i) = \gamma \hat{z}_i + \beta.$$

γ, β обучаются, $\epsilon > 0$ стабилизирует деление.

Статистики считаются **отдельно для каждого признака по батчу**. При обучении выход одного объекта зависит и от других объектов батча.

Ioffe, Szegedy, Batch Normalization, 2015.

Что меняет нормировка признака



Синяя линия — исходный признак. Серая и оранжевая — его сжатая и растянутая копии со сдвигом: $z' = az + b$, $a > 0$.

После вычитания среднего и деления на стандартное отклонение эти копии совпадают. Форма распределения сохраняется. На рисунке показана стандартизация без ϵ и обучаемых γ, β .

BatchNorm: длина и направление весов

Сдвиг b убирается при центрировании. Разделим веса на длину r и направление u :

$$w = ru, \quad r = \|w\| > 0, \quad \|u\| = 1, \quad q_i = u^\top x_i.$$

Тогда $z_i = rq_i$, $\mu_z = r\mu_q$ и $\sigma_z^2 = r^2\sigma_q^2$. После нормировки:

$$\hat{z}_i = \frac{rq_i - r\mu_q}{\sqrt{r^2\sigma_q^2 + \epsilon}} = \frac{q_i - \mu_q}{\sqrt{\sigma_q^2 + \epsilon/r^2}} \approx \frac{q_i - \mu_q}{\sigma_q}.$$

Направление u определяет, какой признак выделяет нейрон. **Длина** r почти сокращается: она одинаково растягивает значения и их стандартное отклонение.

Рассматриваем train-режим, ненулевую дисперсию и малое ϵ относительно дисперсии z . BN нормирует активации, а не сами веса.

Ioffe, Szegedy, Batch Normalization, раздел 3.3, 2015.

Decoupling: признак и его масштаб

После стандартизации применяется обучаемое преобразование:

$$a_i = \gamma \hat{z}_i + \beta.$$

- Поворот вектора w меняет направление u и сам выделяемый признак.
- Изменение длины w почти не меняет нормированный признак.
- Параметр γ регулирует масштаб выхода, β — его средний уровень.

Так обучение направления и настройка амплитуды получают разные параметры. Это уменьшает чувствительность выхода к масштабу предыдущих весов и часто облегчает оптимизацию.

Длина w всё ещё влияет на шаг оптимизатора. Разделение ролей параметров не означает, что они обучаются независимо.

Ioffe, Szegedy, 2015; Arora et al., Theoretical Analysis of Auto Rate-Tuning by Batch Normalization, 2019.

BatchNorm при обучении и применении

Обучение

- Статистики текущего батча.
- Обновление скользящих оценок.
- Градиент проходит и через среднее, и через дисперсию.

При малых батчах оценки могут быть шумными. Перед validation и test в PyTorch вызывают `model.eval()`.

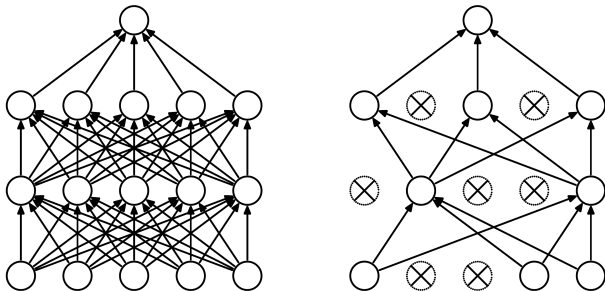
Santurkar et al., How Does Batch Normalization Help Optimization?, 2018.

Применение

- Накопленные статистики train.
- γ, β фиксированы.
- Предсказание не зависит от состава тестового батча.

Dropout

Dropout — способ регуляризации: во время обучения часть активаций случайно заменяют нулями. На каждом прямом проходе выбирается новая маска.



Слева полная сеть, справа один из вариантов после выключения нейронов. Оставшиеся связи используют те же обучаемые веса.

Srivastava et al., Dropout, JMLR, 2014, рис. 1.

Dropout при обучении и применении

Пусть p_{drop} — вероятность выключения, $q = 1 - p_{\text{drop}} > 0$.

Обучение: inverted dropout

$$m_j \sim \text{Bernoulli}(q), \quad \tilde{h}_j = \frac{m_j}{q} h_j.$$

Выключенная активация становится нулём. Оставшиеся увеличиваются, чтобы сохранить средний масштаб:

$$\mathbb{E}[\tilde{h}_j \mid h_j] = h_j.$$

Validation и test

$$\tilde{h}_j = h_j.$$

Случайная маска отключена.
Дополнительное масштабирование не требуется: оно уже учтено при обучении.

В PyTorch этот режим включает `model.eval()`.

Почему Dropout уменьшает переобучение

Нейрон не может постоянно рассчитывать на один и тот же набор соседних признаков: часть из них на текущем проходе отсутствует.

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \mathbb{E}_m [\ell(f_\theta(x_i; m), y_i)].$$

- Оптимизируем качество при разных масках m , используя общие веса θ .
- Это затрудняет запоминание примеров через хрупкие сочетания отдельных признаков.
- Интуитивно обучается множество подсетей, которые разделяют параметры.

Полная сеть при применении лишь приближает усреднение подсетей. Из-за нелинейностей равенство их выходов в общем случае не выполняется.

Какую проблему решает каждый приём

Приём	Основной механизм	Что проверить
ResNet	Сложение входа и поправки, дополнительный путь градиента	Совпадение форм, активация после суммы
BatchNorm	Нормализация по батчу, обучаемый масштаб	Размер батча, статистики и режим применения
Dropout	Случайные маски в обучении, регуляризация	Вероятность выключения, отключение на validation/test

Приёмы добавляют по одному и сравнивают качество на одной и той же validation-выборке.

Оглавление

1) ResNet, BatchNorm и Dropout

2) Свёртки и свёрточные сети (CNN)

3) Рекуррентные сети (RNN)

4) Долгая краткосрочная память (LSTM)

Изображение как вход нейронной сети

Изображение имеет пространственную структуру: близкие пиксели образуют границы, текстуры и части объектов.

- Полносвязный слой может выучить связи между пикселями, но его устройство не использует их соседство явно.
- Один и тот же признак может встретиться в разных частях кадра.
- Для его обнаружения можно применять одно локальное правило во всех положениях.

Идея CNN: локальные связи и совместное использование весов.

Свёртка: одно правило для всех окон

Для изображения с одним каналом, ядра K размера $k \times k$ и единичного шага:

$$Y_{i,j} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{u,v} X_{i+u,j+v} + b.$$

- Берём локальное окно изображения.
- Взвешиваем его значения коэффициентами ядра и складываем.
- Сдвигаем окно, сохраняя те же веса K и смещение b .

Результат во всех положениях образует **карту признака**.

В библиотеках CNN обычно вычисляется кросс-корреляция, как в этой формуле. В математической свёртке ядро переворачивают по обеим осям.

Физический смысл: среднее и производная



Усреднение подавляет быстрые колебания и сглаживает скачок. Разностный фильтр измеряет локальное изменение сигнала и выделяет место скачка.

Производная усиливает и полезные границы, и шум. Сглаживание перед выделением границ уменьшает влияние шума.

Физический смысл фильтров изображения

- **Сглаживание** заменяет значение пикселя взвешенным средним соседей. Мелкие колебания подавляются, резкие переходы размываются.
- **Выделение границ** сравнивает яркости по разные стороны окна. Отклик показывает направление и силу изменения.
- **Повышение резкости** усиливает отличие центра окна от соседей. Локальный контраст возрастает вместе с чувствительностью к шуму.

Оптика распределяет свет от точки по соседним пикселям. Взвешенное усреднение позволяет моделировать такое размытие.

Сглаживание и резкость на фотографии

Исходное изображение



Локальное усреднение



Усиление резкости



Одна обработка ядром 3×3 ; одинаковая шкала яркости $[0, 1]$

Сглаживание убирает мелкие детали вместе с шумом. Повышение резкости усиливает локальный контраст и может усиливать шум.

Изображение: James S. Davis / U.S. Navy, официальный портрет Grace Hopper, public domain. Фильтры применены к одной фотографии.

Направление границы и знак отклика

Исходное изображение



Sobel x: вертикальные края



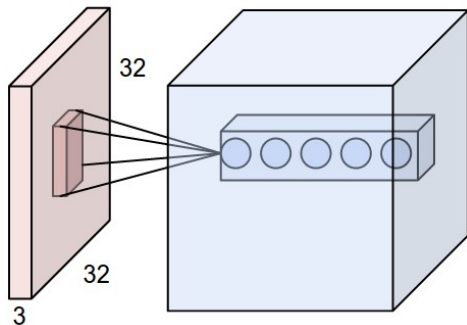
Sobel y: горизонтальные края



K_x выделяет изменение слева направо, то есть преимущественно **вертикальные** границы. K_y выделяет горизонтальные.

Цвет показывает знак и величину отклика, общая шкала одинакова для обеих карт. Ось y направлена вниз.

Каналы и несколько карт признаков



Одно ядро смотрит на **все входные каналы** локального окна:

$$Y_{o,i,j} = b_o + \sum_c \sum_{u,v} K_{o,c,u,v} X_{c,i+u,j+v}.$$

Каждый фильтр имеет свой набор весов и строит отдельную карту признака.

Иллюстрация: Stanford CS231n, Convolutional Neural Networks.

Шаг, дополнение и размер карты

- **Stride** задаёт расстояние между соседними положениями окна. Большой шаг уменьшает пространственный размер выхода.
- **Padding** дополняет изображение по краям и позволяет управлять размером карты.
- **Dilation** увеличивает расстояние между позициями элементов ядра.

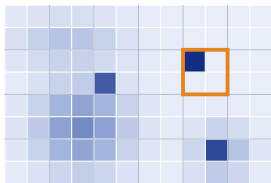
Для одной пространственной оси:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - d(k - 1) - 1}{s} \right\rfloor + 1.$$

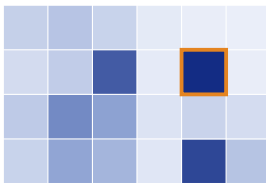
p — дополнение с каждой стороны, s — шаг, d — dilation. Для ширины действует та же формула.

Pooling: локальное сжатие карты

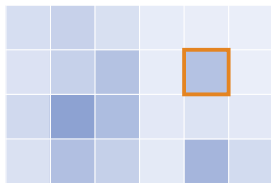
Карта признака



Max pooling



Average pooling



Насыщенность цвета показывает силу отклика

$$y_{i,j} = \max_{(u,v) \in \mathcal{W}_{ij}} x_{u,v}, \quad \bar{y}_{i,j} = \frac{1}{|\mathcal{W}_{ij}|} \sum_{(u,v) \in \mathcal{W}_{ij}} x_{u,v}.$$

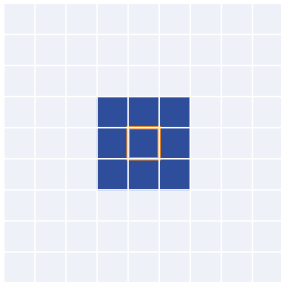
Max pooling сохраняет самый сильный отклик в окне $\mathcal{W}_{ij}$. **Average pooling** усредняет отклики в том же окне.

Каждый канал обрабатывается отдельно. Разрешение уменьшается, точное положение отклика теряется. Обучаемых весов нет.

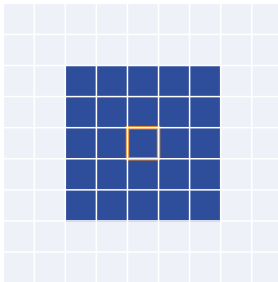
Global Average Pooling (GAP) усредняет карту целиком, оставляя по одному значению на канал.

Рецептивное поле растёт с глубиной

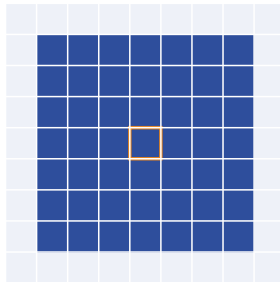
Ранний слой



Промежуточный слой



Глубокий слой

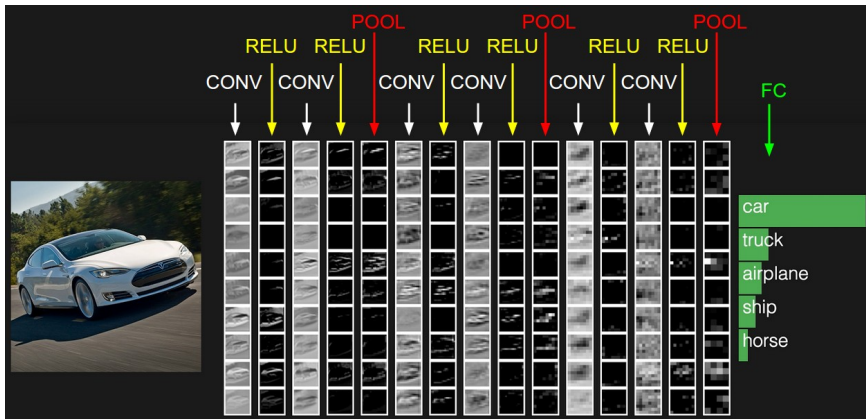


Более глубокий признак зависит от большей области изображения

Каждый следующий слой объединяет соседние признаки предыдущего. Поэтому глубокий признак зависит от большей области исходного изображения.

Показана теоретическая область зависимости. Реальный вклад пикселей определяется весами и активациями.

CNN: карты признаков по глубине



CONV строит новые карты, ReLU добавляет нелинейность, POOL уменьшает пространственный размер. Последний слой выдаёт оценки классов.

Иллюстрация: Stanford CS231n, Convolutional Neural Networks.

BatchNorm и ResNet внутри CNN

Для $X \in \mathbb{R}^{B \times C \times H \times W}$ BatchNorm2d считает статистики **отдельно для каждого канала** по осям B, H, W .

$$\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU}$$

В residual-блоке складываем тензоры одинаковой формы:

$$Y = X + F(X), \quad \text{или} \quad Y = P(X) + F(X).$$

Если меняются каналы или размер карты, проекция P приводит вход к форме выхода обучаемой ветви.

Сдвиг изображения: что сохраняет CNN

Один и тот же фильтр применяется везде. При сдвиге входа сдвигается и карта откликов:

$$f(T_{\Delta}X) = T_{\Delta}f(X).$$

Это **эквивариантность к сдвигу**.

- Формула точна при единичном шаге на бесконечной сетке или вдали от границ.
- Padding, обрезка и увеличение шага ограничивают это свойство.
- Pooling и GAP могут уменьшать чувствительность итогового ответа к положению объекта.

Устойчивость к поворотам, масштабу и смещениям зависит также от данных и аугментаций.

Оглавление

1) ResNet, BatchNorm и Dropout

2) Свёртки и свёрточные сети (CNN)

3) Рекуррентные сети (RNN)

4) Долгая краткосрочная память (LSTM)

Рекуррентная нейронная сеть

Хотим построить модель для обработки последовательности

Рекуррентная нейронная сеть

Хотим построить модель для обработки последовательности

- Сделаем так, чтобы модель на каждом шаге генерировала вектор скрытого состояния h_t , агрегирующий всю полученную ранее информацию

Рекуррентная нейронная сеть

Хотим построить модель для обработки последовательности

- Сделаем так, чтобы модель на каждом шаге генерировала вектор скрытого состояния h_t , агрегирующий всю полученную ранее информацию
- Вектор h_t должен зависеть от предыдущего состояния h_{t-1} и от нового входного слова x_t .

Рекуррентная нейронная сеть

Хотим построить модель для обработки последовательности

- Сделаем так, чтобы модель на каждом шаге генерировала вектор скрытого состояния h_t , агрегирующий всю полученную ранее информацию
- Вектор h_t должен зависеть от предыдущего состояния h_{t-1} и от нового входного слова x_t .
- Пусть $h_t = f(W_{xh}x_t + W_{hh}h_{t-1})$. Матрицы W_{xh} и W_{hh} обучаются на корпусе текстов и не меняются от шага к шагу, f – нелинейная функция активации. Таким образом, матрицы агрегируют старую информацию, помещенную в вектор h_{t-1} , и новую информацию из слова x_t . Добились чего хотели.

Рекуррентная нейронная сеть

Хотим построить модель для обработки последовательности

- Сделаем так, чтобы модель на каждом шаге генерировала вектор скрытого состояния h_t , агрегирующий всю полученную ранее информацию
- Вектор h_t должен зависеть от предыдущего состояния h_{t-1} и от нового входного слова x_t .
- Пусть $h_t = f(W_{xh}x_t + W_{hh}h_{t-1})$. Матрицы W_{xh} и W_{hh} обучаются на корпусе текстов и не меняются от шага к шагу, f – нелинейная функция активации. Таким образом, матрицы агрегируют старую информацию, помещенную в вектор h_{t-1} , и новую информацию из слова x_t . Добились чего хотели.
- Если хотим выдавать что-нибудь на каждом шаге, можем добавить выход $o_t = f(W_{ho}h_t)$. Матрица W_{ho} обучаемая и не меняется в зависимости от шагов.

Рекуррентная нейронная сеть

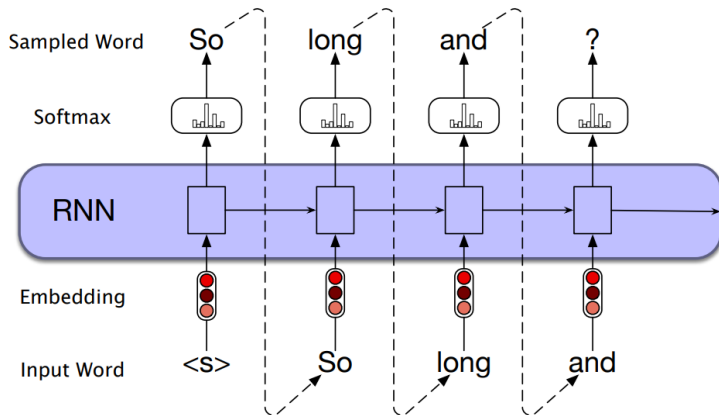


Рис.: Генерация текста с помощью RNN. Сгенерированное слово идет на вход следующего шага.

Рекуррентная нейронная сеть

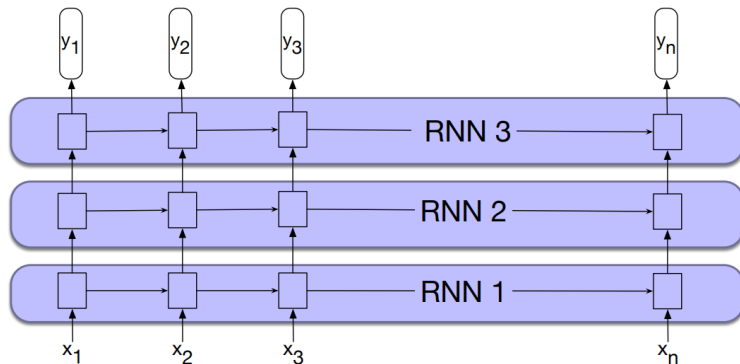


Рис.: RNN слои можно стакать друг на друга. Более нелинейное вникание в контекст.

Рекуррентная нейронная сеть

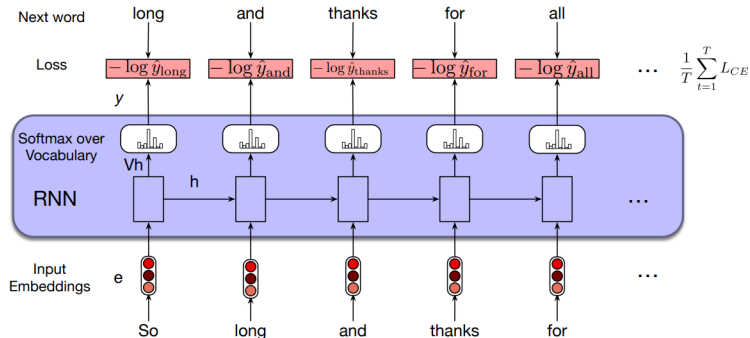


Рис.: Обучение RNN в случае генерации текста. Кросс-энтропийная функция потерь $L_{CE} = -\sum_{w \in V} y_t(w) \log \hat{y}_t(w) = -\log \hat{y}_t(w_{t+1})$.
Здесь $y_t(w) = 1$ только для истинного следующего слова w_{t+1} , иначе равно нулю.

Рекуррентная нейронная сеть

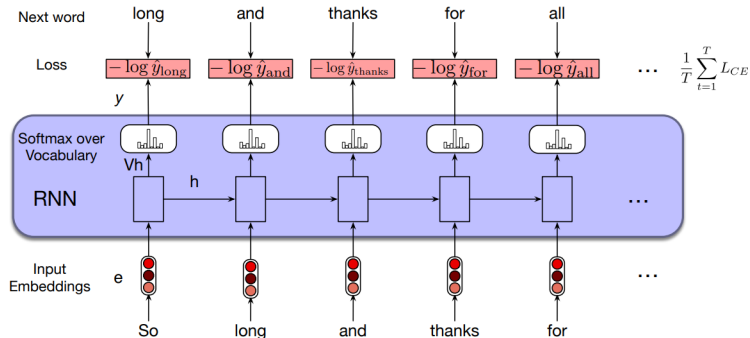


Рис.: Обучение RNN в случае генерации текста. Кросс-энтропийная функция потерь $L_{CE} = -\sum_{w \in V} y_t(w) \log \hat{y}_t(w) = -\log \hat{y}_t(w_{t+1})$.
Здесь $y_t(w) = 1$ только для истинного следующего слова w_{t+1} , иначе равно нулю.

Проблема! Градиенты затухают.

Оглавление

- 1) ResNet, BatchNorm и Dropout
- 2) Свёртки и свёрточные сети (CNN)
- 3) Рекуррентные сети (RNN)
- 4) Долгая краткосрочная память (LSTM)**

Долгая краткосрочная память (LSTM)

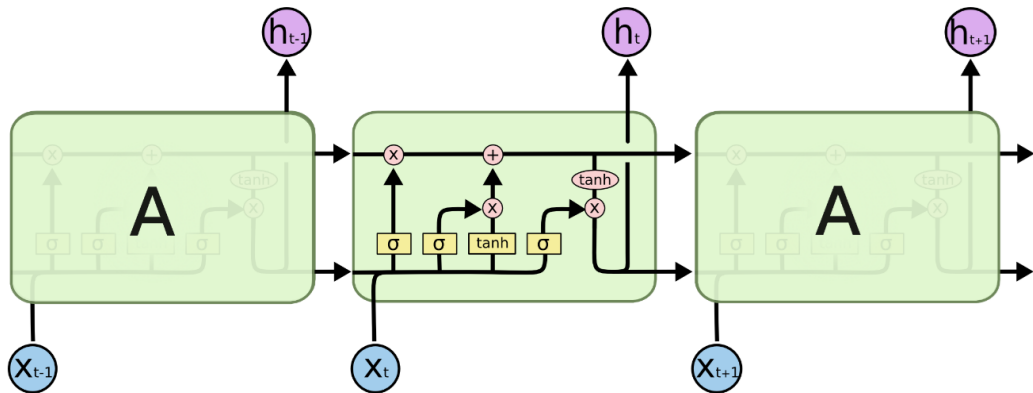


Рис.: Архитектура LSTM.

Долгая краткосрочная память (LSTM). Главная идея

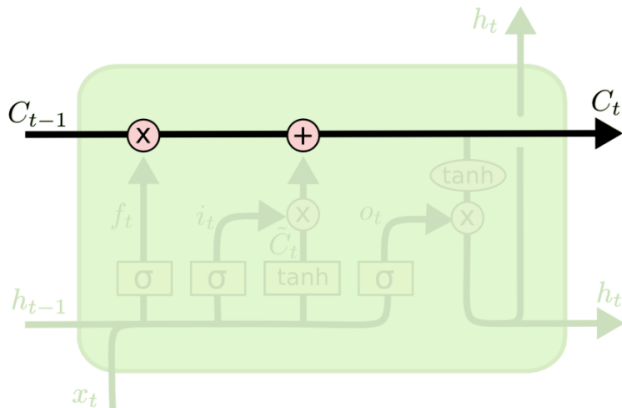
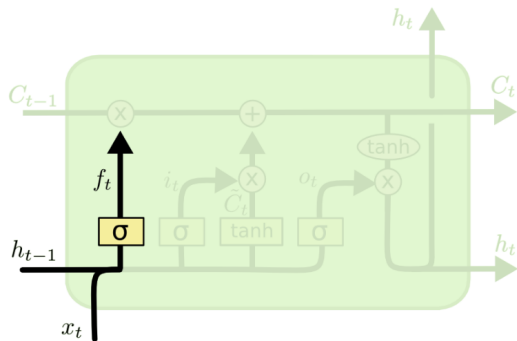


Рис.: Ключевое отличие от обычной RNN – напрямую протекающая через модель информация.

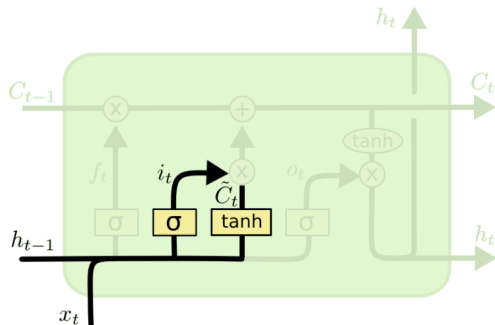
Долгая краткосрочная память (LSTM). Forget gate



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Рис.: Архитектура LSTM модели. Forget gate решает, какую информацию модель должна забыть. Можно, например, забыть информацию про гендер из предыдущих токенов, если контекст изменился и слова относятся к другому человеку/объекту.

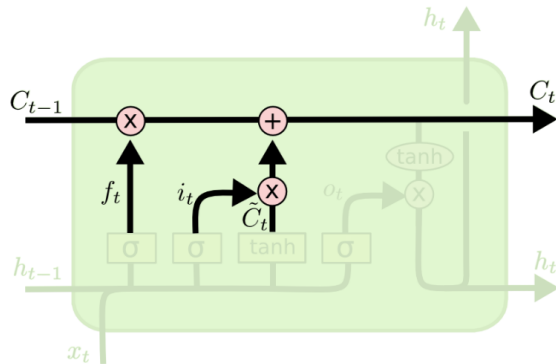
Долгая краткосрочная память (LSTM). Input gate



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Рис.: Input gate решает, какую информацию мы обновляем. Например, захотели обновить информацию о гендере, чтобы поставить правильные окончания/местоимения в новом контексте

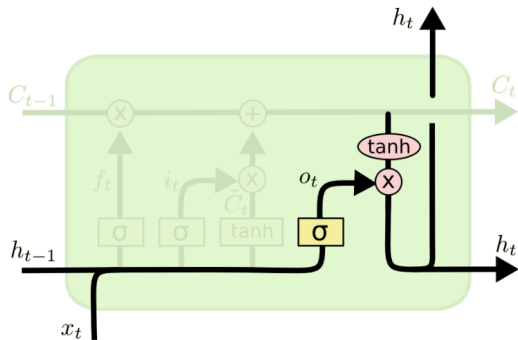
Долгая краткосрочная память (LSTM)



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Рис.: Обновление состояния в соответствии с input gate и forget gate.

Долгая краткосрочная память (LSTM). Output gate



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Рис.: Пробрасываем информацию из вектора состояния C и агрегируем ее с тем, что посчитает важным для этого конкретного выхода h output gate.

RNN и LSTM: общие веса и контекст

Модель	Где общие веса	Как используется контекст
RNN	На разных шагах последовательности	Контекст сжат в скрытое состояние h_t
LSTM	На разных шагах, включая вентили	Память c_t обновляется через управляемую сумму

Выбор архитектуры задаёт предположения о структуре данных. Во всех случаях веса учатся по функции потерь с помощью backprop.

Литература

- He et al. Deep Residual Learning for Image Recognition, 2015/2016.
- Ioffe, Szegedy. Batch Normalization, 2015.
- Srivastava et al. Dropout, 2014.
- Goodfellow, Bengio, Courville. Deep Learning, главы о свёрточных и рекуррентных сетях.
- Stanford. CS231n: Convolutional Neural Networks.
- Jurafsky, Martin. Speech and Language Processing.
- Christopher Olah. Understanding LSTM Networks, 2015.